ASSIGNMENT-16.3

Name: porika manikanta

HT. No: 2303A51271

Batch: 8

Task 1: Hospital Management Database

Scenario: Design a relational database for a Hospital Management System using Python and SQLite. The schema includes Doctors, Patients, and Appointments tables with proper constraints and foreign keys.

Tables: Doctors, Patients, Appointments

Queries Generated:

• List all appointments for a specific doctor.

• Retrieve patient history by patient ID.

• Count total patients treated by each doctor.

Code & Output:

```python
import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.executescript("""
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    specialization TEXT,
    phone TEXT UNIQUE
);

CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    dob DATE,
    gender TEXT CHECK(gender IN ('M', 'F', 'Other')),
    phone TEXT UNIQUE
);

CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date DATE NOT NULL,
    notes TEXT,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);

INSERT INTO Doctors (name, specialization, phone) VALUES
    ('Dr. Smith', 'Cardiology', '9001'),
    ('Dr. Jones', 'Neurology', '9002');

INSERT INTO Patients (name, dob, gender, phone) VALUES
    ('Alice', '1990-05-10', 'F', '8001'),
    ('Bob', '1985-03-22', 'M', '8002'),
    ('Carol', '2000-11-01', 'F', '8003');

INSERT INTO Appointments (doctor_id, patient_id, appointment_date, notes) VALUES
    (1, 1, '2025-01-10', 'Routine checkup'),
    (1, 2, '2025-01-12', 'Follow-up'),
    (2, 3, '2025-01-15', 'Initial consultation'),
    (1, 3, '2025-01-20', 'ECG Test');
""")

print("Appointments for Dr. Smith:")
cursor.execute("""
    SELECT a.appointment_id, p.name, a.appointment_date, a.notes
    FROM Appointments a
    JOIN Patients p ON a.patient_id = p.patient_id
    WHERE a.doctor_id = 1
""")
for row in cursor.fetchall():
    print(row)

print("\nHistory for Patient ID 3:")
cursor.execute("""
    SELECT p.name, d.name, a.appointment_date, a.notes
    FROM Appointments a
    JOIN Patients p ON a.patient_id = p.patient_id
    JOIN Doctors d ON a.doctor_id = d.doctor_id
    WHERE a.patient_id = 3
""")
```

```
Appointments for Dr. Smith:
(1, 'Alice', '2025-01-10', 'Routine checkup')
(2, 'Bob', '2025-01-12', 'Follow-up')
(4, 'Carol', '2025-01-20', 'ECG Test')

History for Patient ID 3:
('Carol', 'Dr. Jones', '2025-01-15', 'Initial consultation')
('Carol', 'Dr. Smith', '2025-01-20', 'ECG Test')

Total Patients per Doctor:
('Dr. Smith', 3)
```

Observation: The schema was successfully normalized with foreign key constraints. SQL JOIN queries efficiently retrieved appointment lists and patient histories. GROUP BY with COUNT gave per-doctor patient totals.

Task 2: Online Shopping Database

Scenario: Design a database for an E-commerce platform with Users, Products, Orders, and OrderDetails tables. Generate analytical queries for business insights.

Tables: Users, Products, Orders, OrderDetails

Queries Generated:

- Retrieve all orders by a user.
- Find the most purchased product.
- Calculate total revenue in a given month.

Code & Output:

```python
import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.executescript("""
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);

CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    price REAL NOT NULL CHECK(price >= 0),
    stock INTEGER DEFAULT 0
);

CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    order_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

CREATE TABLE OrderDetails (
    detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK(quantity > 0),
    unit_price REAL NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

INSERT INTO Users (name, email) VALUES
    ('Alice', 'alice@mail.com'),
    ('Bob', 'bob@mail.com');

INSERT INTO Products (name, price, stock) VALUES
    ('Laptop', 75000, 10),
    ('Phone', 25000, 50),
    ('Headset', 3000, 100);

INSERT INTO Orders (user_id, order_date) VALUES
    (1, '2025-03-05'),
    (1, '2025-03-18'),
    (2, '2025-03-20');

INSERT INTO OrderDetails (order_id, product_id, quantity, unit_price) VALUES
    (1, 1, 1, 75000),
    (1, 3, 2, 3000),
    (2, 2, 1, 25000),
    (3, 2, 2, 25000),
```

```
Orders by Alice:
(1, '2025-03-05', 'Laptop', 1, 75000.0)
(1, '2025-03-05', 'Headset', 2, 3000.0)
(2, '2025-03-18', 'Phone', 1, 25000.0)

Most Purchased Product:
('Headset', 3)

Total Revenue in March 2025:
159000.0
```

Observation: The e-commerce schema demonstrated multi-table JOIN operations. Aggregate functions (SUM, COUNT) with GROUP BY enabled revenue calculation and product popularity analysis. Normalization prevented data redundancy.

Task 3: Library Management Database

Scenario: Design a schema for a Library Management System with Books, Members, and Loans tables. Implement indexing for faster query execution.

Tables: Books, Members, Loans

Indexing Strategy: Indexes created on book_id, member_id, and loan/return dates.

Queries Generated:

• Retrieve all books currently issued (return_date IS NULL).

• Find overdue books (loan date > 30 days ago).

• Count number of books loaned by each member.

Code & Output:

```python
import sqlite3
from datetime import date, timedelta

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.executescript("""
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    isbn TEXT UNIQUE,
    copies INTEGER DEFAULT 1
);

CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE
);

CREATE TABLE Loans (
    loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    loan_date DATE NOT NULL,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);

CREATE INDEX idx_loans_book ON Loans(book_id);
CREATE INDEX idx_loans_member ON Loans(member_id);
CREATE INDEX idx_loans_dates ON Loans(loan_date, return_date);
""")

today = date.today()
overdue = (today - timedelta(days=35)).isoformat()
recent = (today - timedelta(days=5)).isoformat()

cursor.executescript(f"""
```

```
Orders by Alice:
(1, '2025-03-05', 'Laptop', 1, 75000.0)
(1, '2025-03-05', 'Headset', 2, 3000.0)
(2, '2025-03-18', 'Phone', 1, 25000.0)

Most Purchased Product:
('Headset', 3)

Total Revenue in March 2025:
```

Observation: The library schema used julianday() for date arithmetic to detect overdue books. Composite indexes on date columns significantly improved query performance. NULL-check conditions correctly identified currently issued books.

Task 4: Query Optimization

Scenario: Compare an inefficient subquery-based SQL query with an optimized JOIN-based query using EXPLAIN QUERY PLAN to verify performance improvement.

Original Query (Subquery - Slow):

```
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');
```

Optimized Query (JOIN - Fast):

```
SELECT b.* FROM Books b JOIN Authors a ON b.author_id = a.author_id WHERE a.country =
'UK';
```

Code & Output:

```python
import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.executescript("""
CREATE TABLE Authors (
    author_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    country TEXT
);

CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author_id INTEGER,
    FOREIGN KEY (author_id) REFERENCES Authors(author_id)
);

CREATE INDEX idx_books_author ON Books(author_id);
CREATE INDEX idx_authors_country ON Authors(country);

INSERT INTO Authors (name, country) VALUES
    ('Tolkien', 'UK'),
    ('Rowling', 'UK'),
    ('Hemingway', 'USA');

INSERT INTO Books (title, author_id) VALUES
    ('Lord of the Rings', 1),
    ('Harry Potter', 2),
    ('The Old Man', 3),
    ('The Hobbit', 1);
""")

print("Original Query (Subquery):")
cursor.execute("""
    SELECT * FROM Books
    WHERE author_id IN (
        SELECT author_id FROM Authors WHERE country = 'UK'
    )
""")
for row in cursor.fetchall():
    print(row)
```

```
Students in Data Structures:
('Alice', '23CS01', 'Data Structures')
('Bob', '23CS02', 'Data Structures')
('Carol', '23CS03', 'Data Structures')

Faculty Teaching More Than 2 Courses:
('Prof. Rao', 3)

Courses by Registration Count:
('Data Structures', 3)
('DBMS', 2)
('Algorithms', 1)
('Calculus', 1)
('Operating Systems', 1)
```

Observation: Both queries return identical results — all books by UK authors. The optimized JOIN query avoids repeated subquery execution and leverages indexes more effectively.

EXPLAIN QUERY PLAN confirms the JOIN uses the index directly, making it significantly faster on large datasets.

Task 5: University Course Registration

Scenario: SR University needs a course registration system for students enrolling in different courses under faculty supervision.

Tables: Students, Courses, Faculty, Registrations

Relationships:

• Each student can register for multiple courses.

• Each course is taught by one faculty member.

Queries Generated:

• List all students enrolled in a specific course.

• Find faculty members teaching more than 2 courses.

• Retrieve courses with the highest number of registrations.

Code & Output:

```python
import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.executescript("""
CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    department TEXT
);

CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    credits INTEGER DEFAULT 3,
    faculty_id INTEGER,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);

CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    roll_no TEXT UNIQUE NOT NULL,
    email TEXT UNIQUE
);

CREATE TABLE Registrations (
    reg_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    reg_date DATE DEFAULT CURRENT_DATE,
    UNIQUE (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

INSERT INTO Faculty (name, department) VALUES
    ('Prof. Rao', 'CSE'),
    ('Prof. Meera', 'IT'),
    ('Prof. Arjun', 'MATH');

INSERT INTO Courses (title, credits, faculty_id) VALUES
```

```
Students in Data Structures:
('Alice', '23CS01', 'Data Structures')
('Bob', '23CS02', 'Data Structures')
('Carol', '23CS03', 'Data Structures')

Faculty Teaching More Than 2 Courses:
('Prof. Rao', 3)

Courses by Registration Count:
('Data Structures', 3)
('DBMS', 2)
('Algorithms', 1)
('Calculus', 1)
('Operating Systems', 1)
```

Observation: The university registration schema successfully enforced a UNIQUE constraint on (student_id, course_id) to prevent duplicate registrations. HAVING clause filtered faculty with more than 2 courses. ORDER BY with COUNT produced a ranked list of courses by popularity.